ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/263611839

Simulation at Extreme-Scale: Co-Design Thinking and Practices

Article *in* Archives of Computational Methods in Engineering · March 2014

DOI: 10.1007/s11831-014-9095-y

CITATIONS
6

READS
63

1 author:

Rong Tian
Chinese Academy of Sciences

42 PUBLICATIONS **488** CITATIONS

SEE PROFILE

All content following this page was uploaded by Rong Tian on 09 July 2017.

The user has requested enhancement of the downloaded file.

Arch Computat Methods Eng (2014) 21:39–58
DOI 10.1007/s11831-014-9095-y

Simulation at Extreme-Scale: Co-Design Thinking and Practices

Rong Tian

Received: 6 January 2013 / Accepted: 3 July 2013 / Published online: 1 February 2014
© CIMNE, Barcelona, Spain 2014

Abstract The current trend of computer architecture evolving towards exaflop/s is the fast increasing floating point performance (the so-called "free" flops) accompanied by much slowly improving the bandwidth of memory and network. Numerical simulation would undergo the challenge posed by the unbalanced increase in the compute power and the capability of data movement. In this paper, after reviewing the challenges of hardware and software in moving towards exascale computing, we present co-design thinking for selecting, optimizing, and developing a numerical algorithm and a simulation tool to meet the challenge of simulation at extreme scale. Examples are presented to demonstrate the new way of thinking and its effectiveness on the emerging architecture.

Keywords Exascale computing · Manycore · Performance gap · Co-design · Hardware-efficient numerical algorithms · Scalability · Fault tolerance

1 Introduction

The work is motivated by the challenge in fully utilizing high floating point performance of the emerging manycore architecture. A fundamental trend of computer architecture evolving towards exaflop/s [10^{18} floating point operations (flops) per second] is the fast increasing floating point performance (so-called "free flops") accompanied by much slowly increasing the capability of data movement. Numerical simulation is facing a challenge posed by the unbalanced increase in the compute power and the bandwidth. The tomorrow's numerical simulation is likely to be subject to a bandwidth constraint instead of a performance/flops constraint. A numerical method of PDEs should be designed to request more flops per unit data movement so that the "free flops" are not sitting idle and wasted [1–5].

We introduce the so-called "performance gap" of the today's supercomputers, which refers to the difference of the performance that can be actually utilized by real S&E applications and the LINPACK benchmark test [6]. In Sect. 3, by viewing the trend of hardware change, we point out that the performance gap may become wider if the current numerical algorithms do not change to catch up with the emerging architecture. Following the trend of hardware change, tomorrow would see a paradigm shift in defining a "good" numerical algorithm, see the discussion in Sect. 4. In Sect. 5, some co-design ideas are proposed, including the missing metric for evaluating the performance of an application code, the better hardware from the viewpoint of domain applications, a numerical algorithm with tunable arithmetic intensity, the mixed precision computing strategy, high scalability and fault tolerance of simulation codes, and etc. Based on our research work, those ideas are tried out and demonstrated in Sects. 7, 8, 9, 10. Conclusions are drawn in the last section.

2 Extreme-Scale Computing

According to the Top500 ranking of November of 2011, the fastest computer is Titan installed in Oak Ridge National Laboratory in USA, whose LINPACK speed is 17.59 petaflop/s. As of June 2013, Chinese Tianhe-II is ranked the fastest with a LINPACK speed of 30.65 petaflop/s. One petaflop/s is 10^{15} floating point operations (flops) per second. The next leap, an exaflop/s, is 1,000 petaflops, which is the current goal of computer scientists.

In 2010, the U.S. Department of Energy (DOE) proposed the first prototype of the exaflop/s system and planned its

R. Tian (✉)
Institute of Computing Technology, Chinese Academy of Science, Kexueyuan Nanlu 6, Haidian 100190, Beijing, China
e-mail: rongtian@ncic.ac.cn

Table 1 Prototype of exaflop/s systems and current petaflop/s systems

Design item	Titan (2011)	Tianhe-II (2013)	2010 design(2018–2019)	2012 design (2020–2022)
Peak performance	27 Pflop/s	54.9 Pflop/s	1 Eflop/s	1.2 Eflop/s
Power	8.3 MW (2 Gflops/W)	17.8 MW (1.935 Gflops/W)	~20 MW (50 Gflops/W)	~20 MW (50 Gflops/W)
System memory	710 TB (32 + 6)*18688)	1.4 PB (64 + 3*8)*16000)	10 PB	32-64PB
Node Performance	1452 Gflop/s (141 + 1311)	3431 Gflop/s (422 + 3009)	10 Tflop/s	1.2 or 15 Tflop/s
Node mem.bdw	232 GB/s (52 + 180)	272 GB/s(?) (32(?) + 240)	400 GB/s	2−4 TB/s
Node concurrency	16 CPU + 2688 Nvidia cores	24 CPU + 3*57 Xeon Phi cores	O (1 K)	O (10 K) or O (1 K)
Node total network bdw	8 GB/s	6.35 GB/s (MPI broadcast)	50 GB/s	200–400 GB/s
System nodes	18,688	16,000	O (1 M)	O (100 K) or O (1 M)
System concurrency	560640 (299008 AMD cores + 261632 Nvidia cores)	3120000 (384000 Ivy Bridge cores + 2736000 Xeon Phi cores)	O (1B)	O (1B)
MTTF	?	?	O (<1 day)	O (<1 day)

deployment in 2018 [2]. The design has been adjusted in 2012 and the deployment timeframe is postponed from 2018 to 2020–2022 according to [7–9]. Although the prototype of an exaflop/s system would change with time, the two designs may provide a basic image of the future exaflop/s system. We list up Titan, Tianhe-II and the two exascale system designs in Table 1.

The baseline of the two designs is 20 MW power consumption and a budget of USD 200M [1–3,10,11]. In the table, the "change ratio" for each design item can be estimated. It is observed that the 2012 design keeps the same the power consumption (physical constraint) and the system concurrency, significantly increases system memory, memory bandwidth and network bandwidth, but considers a slowly increase or even a decrease in the node performance. Compared with the 2010 design, the 2012 plan pays attention to the balance of the floating point performance and the capability of data movement, which is targeting at a more balanced system.

Nevertheless, the 2012 design would rely on aggressive technology evolution in the next <10 years. For examples, compared with the current techniques used for Titan, the memory bandwidth is expected to increase 8–17 times and the network bandwidth to increase 25–50 times. Compared with Titan, we expect the exaflop/s system a 37 times of increase in peak performance while only a 2.4 times of increase in power consumption. If Titan still leaves room for improvement, Tianhe-II clearly shows a need for dramatic technique advancement: we already reached the boundary of 20 MW power consumption (The total power consumption of Tianhe-II including cooling is 24 MW) and we would not be able to build the exaflop/s system following the currently incremental progress of technology.

On the other hand, domain experts' prime concern is that the power constraint may leave little space for hardware designers to consider thoroughly the characteristics of real S&E applications. A quick question hence is *would the system be efficient enough for real S&E applications?*

3 S&E Applications at Extreme-Scale

The promise of computational science in the extreme-scale computing era is to reduce and decompose macroscopic complexities into microscopic simplicities with the expense of high spatial and temporal resolution of computing. "direct numerical simulation (DNS)", which in a broad sense means computing with little parameterization and phenomenological models, should be the fundamental approach towards the transformative and predictive simulation.

The US National Center for Computational Science at Oak Ridge National Laboratory selected 6 codes for Titan as early exascale computing codes [12]. Those codes are to a certain extent representative of the characteristics of the S&E applications at extreme-scale.

(1) S3D (combustion) [13]: 8th FD, 4th explicit RK integrator, DDM with static load balance, non-blocking message, structured grid;
(2) CAM-SE (climate) [14]: FD (vertical) and SE (horizontal), time-split fully explicit form; 2nd N-stage RK method, cubed-sphere grid;
(3) DENOVO (nuclear energy) [121]: to solve the radiation transport problem for advanced nuclear reactor design, to solve the linear Boltzmann equation in six dimensions (3-space, 2-angle, 1-energy), primary algorithms: the discrete ordinates method, the 80–99 % execution time is spent on a 3D sweeping algorithm with a matrix-vector multiply kernel, GMRES linear solver and various

eigensolvers (Arnoldi etc), scaled up to 200 K cores on Jaguar, structured grid, 10 % peak performance, FD, semi implicit;

(4) LAMMPS (classic MD, bio) [15]: fully explicit;
(5) PFLOTRAN [earth science (CO_2 sequestration)] [16]: fully implicit, scaled up to 65 K cores, 2B degrees of freedom (dofs), FD;
(6) WL-LSMS (first principle calculation) [17]: Density function calculation, Monte–Carlo, sustained performance similar to Linpack.

Those large-scale codes are all characteristic of pursuing an unprecedentedly high resolution of simulation in a *direct* way aiming at avoiding free parameters and reducing phenomenological modeling at subscale. Taking in mind finite difference stencil computation in S3D, CAM-SE, DENOVO, PFLOTRAN and matrix vector multiplication in PFLOTRAN and DENOVO, those real applications mostly feature ***memory intensive***.

4 "Performance Gap" of Today's Supercomputers

In the HPC community, it is commonly acknowledged that even a finite element code delivers nearly linear speedup, the percent of the peak performance that the code can really utilize is usually rather small. For an implicit method in which core operations are numerical linear algebra solving a system of linear equations, the percent is rarely beyond 20 %. As an example, we estimate in Table 2 the theoretical upper bound of the performance of a matrix vector (MV) multiplication—the core of the most of Krylov subspace methods—in double precision on AMD Opteron 6,274 (Interlagos) CPU, Tesla k20x GPU, and the supercomputer Titan. It can be theoretically estimated is that the performance on the hardware listed is not beyond 10 %, which means only a very small portion of the floating point peak performance can be utilized in real applications, leaving 90+ % underutilized!

On the other hand, the efficiency obtained for the linear algebraic package (LINPACK) benchmark test, a benchmark used to test the performance of a computer [6], is more than 80 % in general. For the Japanese K-computer, the LINPACK test delivers 93 % efficiency. The MV kernel represents the core characteristics of a majority of S&E applications. Clearly, there exists a huge gap between the real S&E application performance and the nominal benchmark performance (Fig. 1) [4,5].

The reason behind the performance gap is not complicated. A large portion of the S&E applications is to solve a system of linear equations and/or the stencil computation. The Krylov subspace methods finally reduced to sparse matrix vector multiplication (spMV). The theoretical upper bound of the "arithmetic to memory ratio" (AMR) of spMV is only 2:1 (measured in flop/word), which is typically a memory intensive algorithm (computing is memory bandwidth bound). However, the LINPACK benchmark test uses a direct method to solve a system of linear equations. Its core algorithm is double precision general matrix multiplication (DGEMM). The AMR is n:1, where n is the size of the sub-matrix, which features computing intensive. The AMRs of the algorithms of the real S&E application and the LINPACK test are fundamentally different. It is the fundamental difference that determines the huge gap of a computer's performance in the real S&E application and in the LINPACK benchmark test.

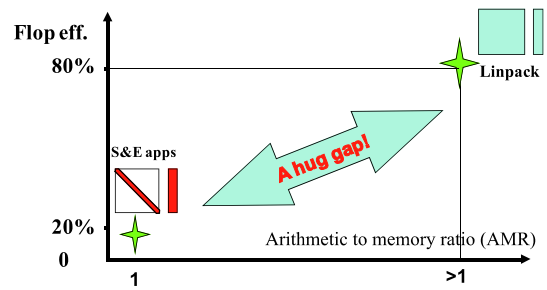

Fig. 1 Performance gap

Table 2 Theoretical upper bound of efficiency of matrix-vector (MV) multiplication [Arithmetic to memory ratio (AMR): 2:1 in flop/word] in double precision

Hardware	AMD Opteron 6274	GPU (Tesla k20x)	Titan
Performance (Gflop/s)	141	1,311	1,452
Memory bandwidth (GB/s)	52	180	232
Upper bound of efficiency	10 %	3.4 %	4 %

5 Tomorrow's Hardware Change

Figure 2 shows the performance of two nVidia GPUs, Tesla C2050 and Tesla C1060 [18]. The specifications of the two GPUs are list in Table 3. The left handside figure is the test results for double precision general matrix multiplication using the DGEMM subroutine of cuBLAS3.1, which is the core algorithm of LINPACK benchmarking. The right handside is tested using 13 real S&E applications (which are mostly finite element/difference methods). The core algorithm of those S&E applications is the sparse matrix-vector multiplication spMV [18]. The tests are originally to highlight the good performance of the manycore GPUs over the multicore CPU. Based on the same raw data, we, from a different viewpoint, make a quick comparison between the

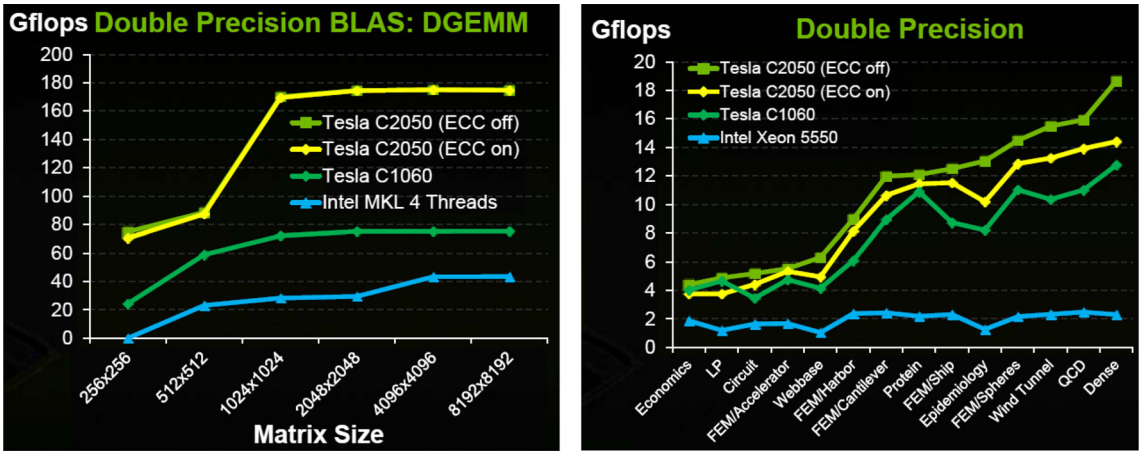

Fig. 2 Performance tests of Tesla C2050 and Tesla C1060 GPUs [18] (Courtesy of nVidia (R))

Table 3 nVidia Tesla C-series GPUs: C2050 and C1060

GPU	Power	Performance	Memory bandwidth
C1060	188 W	78 Gflop/s	102 GB/s
C2050	247 W	515 Gflop/s	144 GB/s

two GPUs and to get a sense of the tomorrow's manycore hardware.

The floating point peak performance of C2050 is 515 Gflop/s in double precision, while that of C1060 is 78 Gflop/s. The former is 6.6 times fast. If asking *which hardware is better*, the answer based on the experience of the PC times, very likely, quickly goes the former (*the faster, the better*).

Now let us focus on the performance difference of the two GPUs in each figure. From Fig. 2, it is seen that the performance of C2050 and C1060 for DGEMM in the left handside figure are significantly different, whereas their performance are quite close for spMV in the right handside figure. C2050 is about two times fast compared with C1060 for DGEMM whereas only 40 % faster than C1060 for spMV.

The performance difference lies in the following fact. DGEMM is computing intensive, whose performance is determined by the floating point performance of hardware, whereas spMV is memory intensive, whose performance is determined by the memory bandwidth of hardware. The memory bandwidth of C2050 (144G B/s) is 46 % wider than C1060 (102 GB/s). This poses a hardware upper limit on the performance improvement of spMV on C2050. Therefore, the performance improvement of C2050 over C1060 for spMV cannot exceed 46 %. This explains the 40 % performance improvement for the real S&E applications, although C2050 is 6.6 times fast compared with C1060. If taking into considerations power consumption and price, C1060 is not too bad. If in terms of floating point efficiency—the percent of the floating point peak performance that can be effectively used—C1060 is 14 % (11/78), whereas C2050 only 3 % (16/515), C1060 is reversely more efficient.

Obviously, the *memory intensive* S&E applications do not simply benefit from the pure increase in the floating point performance. It is the balance between the floating point performance and the memory access capability that predetermines the performance gain of the real S&E applications. The core algorithm of the most S&E applications is largely memory intensive (stencil computation and/or spMV). In consideration of the current trend of hardware change, if no change is made to the key numerical algorithms, the waste of the floating point capability seems unavoidable.

In short, we believe that simply increasing the number of cores is not necessarily a good news to the most of S&E applications and reversely it tends to widen the existing performance gap; a revisit to the key numerical algorithms is critically necessary to catch up with the hardware change.

6 "Good" Algorithms on Emerging Manycore Architectures

In the PC/single core times, we usually assume flop is expensive. As such, the "best" practice having been followed is to trade frequent memory access for reducing and saving flops; the philosophy of code/algorithm optimization is also to reduce the number of flops. However, following the current trend of hardware change, flop is becoming the next round of "free lunch", whereas data movement becomes the new bottleneck because the energy consumed on data moment is 100X that on flop [3]. Hence, from the viewpoint of hardware efficiency and energy effectiveness, the more flops per unit data movement, the merrier. For the future's exascale system, the S&E application should adopt a numerical algorithm that can maximize the number of flops per unit data movement

Table 4 Influence of numerical algorithm on floating point efficiency

Computing intensity of S&E applications (flop/word)	Hardware's AMR (flop/word)		
	AMD Opteron 6274 (21:1) (%)	Tesla k20x GPU (58:1) (%)	Titan (50:1) (%)
1:1	5	2	2
10:1	47	17	20
20:1	95	34	50
58:1	100	100	100

to take advantage of the "free flop". This is in perfect contrast with the programming habit and the way of thinking in the past. The algorithm that is highly accurate but was deemed to be expensive in the PC/single core times might be reversely a good candidate on the emerging computer architecture; tomorrow highly possibly would see a paradigm shift in defining a "good" numerical algorithm.

7 Co-Design Thinking from a Respective of Numerical Algorithm

The performance or the flop efficiency of a numerical algorithm is defined to be the ratio of the arithmetic intensity (AI) of algorithm to the arithmetic to memory ratio (AMR) of hardware. The AI of algorithm is the ratio of the number of flops to the times of memory access of an application code while the AI of hardware is the ratio of the peak performance to the memory bandwidth of hardware. Therefore, for a given hardware, the performance or the flop efficiency of application is closely related to its AI. Taking AMD Opetron 6,274 (Interlagos) CPU, Tesla k20x GPU, and Titan as examples, we show in Table 4 that by changing the AI (in the unit of "flop/word") of application, the performance or the flop efficiency of application can be improved accordingly; once the AI of algorithm is beyond 58:1, in theory, all the hardware will be 100 % efficient. This theoretical prediction shows that *the change of the AI of algorithm can significantly improve the efficiency of hardware*.

However, the traditional metric for ranking a numerical algorithm is the properties like the order of accuracy, the property of convergence and etc., which are purely mathematical. Little attention has been paid to hardware constraint on the numerical algorithm. For example, in the past one seldom paid attention to the AI of a numerical algorithm. To domain experts, this implies that we may re-visit the key numerical algorithms of domain applications and to re-think their effectiveness on the emerging computer architecture by taking into account the new metrics such as the complexity of data movement, the arithmetic intensity and etc.

In the author's opinion, co-design thinking may be like that:

- For applications: even a linearly speeduped code is not enough; new algorithms have to be designed/invented to utilize the free/redundant flops;
- For hardware: it is not necessarily a good news to simply increase peak flop performance. Alternatively, performance metric has to reflect the balance between performance and the bandwidth of data movement. Single flop performance metric is definitely not the whole story. As a matter of fact, the flop performance oriented LINPCK principle starts to twist the design of supercomputer—widening the existing performance gap—in the manycore times.

  From a viewpoint of numerical algorithm development, we suggest the following co-design ideas:
- to increase arithmetic intensity: to transform from the memory to the computing intensive
  1. Linear methods versus high order methods
  2. Re-computation versus frequent storage access
  3. Arithmetic intensity tunable algorithms (need to redesign numerical algorithms)
- Mixed precision computing: *working with deltas in low precision*, rather than the whole value in high precision [123]
  1. Mixed precision computing acceleration
  2. Mixed precision incremental communication
- Special attention to the simulation code at extreme-scale
  1. It must be head-to-toe parallel. The legacy codes prototyped in the PC/sequential times need to be restructured largely.
  2. It must be fault tolerant. It might be easier for a domain expert to do this than a computer scientist.

  Some of those ideas will be tried out in the following sections.

8 Co-design example I: A new respective on "expensive" methods

In the PC/single core times, we value sparsity: the sparser the matrix, the more efficient; a high order method usually is deemed inefficient as it leads to a denser global matrix.

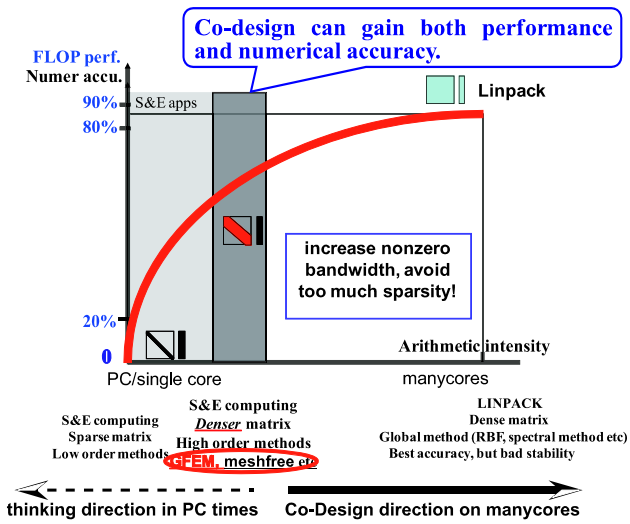

Fig. 3 "Good" algorithms on emerging architecture [19]

However, a moderately dense matrix means more arithmetic operations and potentially more computing intensive. Meanwhile, for a given number of dofs, the dense matrix often is a sign of high approximation accuracy. Therefore, the moderately dense matrix may become a combination of hardware efficiency and high approximation accuracy. This is a "win-win" strategy. As such, on the emerging manycore architecture we may value a numerical method which leads to the moderately dense matrix. This signals a philosophy shift in selecting, optimizing, and defining a "good" numerical algorithm. As illustrated in Fig. 3, the meshfree and GFEM approximation, those "expensive" methods, may deserve a revisit from a prospective of hardware-efficiency [19].

For the same mesh or the same number of dofs, a meshfree approximation offers high accuracy for a large influence radius. Roughly speaking, the number of nodes or dofs is related to memory requirement, whereas the influence radius is related to the number of flops. Therefore, when the influence radius increases, the arithmetic intensity should also increase—for n points in the influence circle, each point is repeatedly used n times. This is to say that the arithmetic intensity in the meshfree approximation is tunable through the influence radius; this is a unique feature of the meshfree method over the standard FEM. The meshfree/GFEM approximation might better fit the emerging manycore computer architecture. We are motivated to develop a GFEM without extra dof.

8.1 A GFEM Without Extra Dof [20]

The fantastic possibilities of the partition of unity (PU) approximation were first elaborated in the partition of unity method (PUM) and the partition of unity finite element method (PUFEM) by Melenk and Babuška [21,22] (and earlier in [23]). The similar idea to the PUM was also introduced in the *hp*-cloud method by Duarte and Oden [24,25]. The *finite element* PU is extensively investigated in the PUFEM [22], the generalized finite element method (GFEM) by Strouboulis and Babuška [26–33] and the GFEM by Duarte et al. [34–38].

The core of the GFEM is a PU approximation (in 1D):

$$u^h(x) = \sum\nolimits_{i\in I} N_i u_i + \sum\nolimits_{i\in I} N_i \sum\nolimits_k \varphi_k^{\mathrm{P_i}} a_{i(k)} \tag{1}$$

where $\sum_i N_i(x) \equiv 1$ form the PU, $\varphi_k^{\mathrm{P_i}}$ is the user-defined local function on patch P_i which is composed of elements surrounding node i, and $a_{i(1)}, a_{i(2)}, \ldots$ are the extra dofs of node i.

The number of dofs per node in the existing GFEM varies with the order of local approximation. Therefore, the increase of the approximation order does not improve its arithmetic intensity. An extra-dof-free and linearly independent enrichment is proposed in [20] and is cited below.

Let P_i^r be a patch composed of elements surrounding node i, where r denotes the size of the patch. The patch size is either the size of nodal support combining $m \geq 1$ layer(s) of elements surrounding node i on a regular mesh or simply the radius of an influence circle at node i on an arbitrary mesh. Node i is referred as "patch star" and index i is solely kept for the patch star and $j\,(j \neq i)$ is used to denote any a non-patch star node on the patch. Node set $\left\{x_k \,|\, x_k \in \mathrm{P}_i^r\right\}$ is a collection of *all* the nodes on P_i^r.

Now let us construct an approximation of $u_i(x)$, $x \in \mathrm{P}_i^r$, among the nodal values of $\left\{x_k \,|\, x_k \in \mathrm{P}_i^r\right\}$ local to P_i^r as

$$u_i(x) = \sum_{k=1}^{n_i} L_k^{\mathrm{P_i^r}}(x)\, u_k, \quad x, x_k \in \mathrm{P}_i^r \tag{2}$$

where $L_k^{\mathrm{P_i^r}}(x)$ is the local function defined with regard to k-th node in the node set which usually also forms a partition of unity, $\sum_{k=1}^{n_i} L_k^{\mathrm{P_i^r}}(x_i) \equiv 1$, n_i is the number of nodes on the patch, and u_k is the conventional nodal unknown.

Use the approximation $u_i(x)$ as a local approximation at node i and directly substitute it for the nodal unknown u_i in the following standard FEM approximation

$$u^h(x) = \sum_{i=1}^{N} N_i(x)\, u_i \tag{3}$$

where N_i is the standard FE shape function, a new approximation then is obtained as

$$u^h(x) = \sum_{i=1}^{N} \left(N_i \left(\sum_{k=1}^{n_i} L_k^{\mathrm{P_i^r}} u_k \right) \right) \tag{4}$$

where N is still the number of nodes on the mesh of the standard FEM (3), and u_k is still the nodal unknown of the standard FEM. Different from the standard FEM, one individual node appears repeatedly in n_i patch-wise local approximations in (4).

In approximation (4), corresponding to the n_i nodes on patch i, each node u_i associated with n_i local functions, which are respectively:

$$\underbrace{N_i L_i^{\mathrm{P_i^r}} u_i}_{\text{patch star node}}, \underbrace{N_1 L_i^{\mathrm{P_1^r}} u_i, N_2 L_i^{\mathrm{P_2^r}} u_i, \cdots, N_{j,j\neq i} L_i^{\mathrm{P_j^r}} u_i, \cdots, N_{n_i} L_i^{\mathrm{P_{n_i}^r}} u_i}_{\text{non-patch star nodes}} \tag{5}$$

where the superscript k in the standard FE shape function N_k, $k = 1, 2, \ldots i, \ \ldots, n_i$, is a local number, which is defined at the nodes of patch i, and $L_i^{\mathrm{P_k^r}}$ is the local function constructed using the nodes (which are partially from patch i) from patch k with regard to node i(which is shared by patches k and i). Noted is that $L_i^{\mathrm{P_k^r}}$ and $L_k^{\mathrm{P_i^r}}$ is not equivalent.

By keeping in mind (5) and re-grouping the terms in (4) by u_i, we obtain

$$u^h(x) = \sum_{i=1}^{N} \left(\sum_{k=1}^{n_i} N_k L_i^{\mathrm{P_k^r}} \right) u_i = \sum_{i=1}^{N} \tilde{N}_i u_i \tag{6}$$

where n_i, the number of nodes on patch P_i^r in (2), now really means the number of patches containing node i,but the two numbers are equal, and $\tilde{N}_i$ denotes the new shape function, which is a summation of n_i terms. This completes the basic idea of the work, which as seen above is quite straightforward.

Comparing (4) with (1), it is immediate that the form (4) is also a PU approximation. However, the new PU approximation has two unique features: (1) it does not include any extra dof; (2) the PU approximation interpolates as long as the patch-wise local approximation interpolates at patch star i, regardless of that it interpolates or not at the non-patch star nodes.

A large patch size r facilitates a high degree of $L^{\mathrm{P}_i^r}$ and hence a high order of the global approximation. This resembles the core feature of the existing GFEM. The new PU approximation does not include any extra dof and the total number of dofs to be solved is invariable with the local function $L^{\mathrm{P}_i^r}$. Both an interpolation and a *selectively interpolating* approximation can be used to construct the local approximation [20]. When an approximation (for example the moving least squares approximation [39]) is used to construct the local approximation, *selectively interpolating* means the approximation should be enforced to pass the patch star, which is easy to satisfy. The resulting new GFEM always interpolates, no matter the local approximation is approximative or interpolative. The more detail can be found in [20].

8.2 Lagrange Interpolation Polynomial Local Approximation

On a structured mesh, the local approximation can be constructed by Lagrange interpolating polynomials. For patch size $r = ch$ (h is the mesh size), the local function $L_k^{\mathrm{P_i^r}}(x)$ can be expressed as Lagrange interpolating polynomials in x direction,

$$L_k^{\mathrm{P_i^{rx}}}(x) = \prod_{j=1, j\neq k}^{n_i} \frac{(\xi - \xi_j)}{(\xi_k - \xi_j)}, \xi = \frac{x - x_i}{h}, \tag{7}$$

where r_x is the patch size in x. In high dimensions the tensor product of the above one-dimensional Lagrange interpolating polynomial can be used. The order of $L_k^{\mathrm{P}_i^{r_x}}$ is solely determined by and increases with n_i.

The RBF and the Selectively Interpolating MLS can be used to construct a local approximation for an unstructured mesh of any dimension [20]. In the study, main attention is paid to its floating point performance on the emerging architecture. The numerical performance of the new GFEM does resemble the existing GFEM; tests are provided to demonstrate its capability of offering a different order of accuracy and convergence for the same number of dofs, i.e., tunable arithmetic intensity.

8.3 Numerical Performance of the New GFEM

The following benchmark problem is considered in 2D

$$\begin{aligned} &\Delta \mathbf{u} = -\mathbf{f} \quad in\ \Omega \\ &\mathbf{u} = 0 \quad on\ \partial\Omega \end{aligned} \tag{8}$$

on a regular domain $\Omega = [0, 1]^2$. The exact solution is taken to be

$$u_x = u_y = xy(1-x)(1-y)\sin(x)\sin(y) \tag{9}$$

Errors are measured by the L_2 and energy norms defined respectively below

$$\begin{aligned} \|u\| &= \left(\int_\Omega \left(u^h(x) - u(x) \right)^2 \mathrm{d}\Omega \right)^{\frac{1}{2}}, \\ \|e\| &= \left(\int_\Omega \left(\nabla u^h(x) - \nabla u(x) \right)^2 \mathrm{d}\Omega \right)^{\frac{1}{2}} \end{aligned} \tag{10}$$

The mesh is taken to be a Cartesian mesh that is refined uniformly for convergence tests (Fig. 4). The patch size is defined by a square nodal patch. In the following, the tensor product Lagrange interpolation polynomial local approximation is employed. $c+2$ point element-wise Gaussian quadrature is used. Test results are shown in Fig. 5.

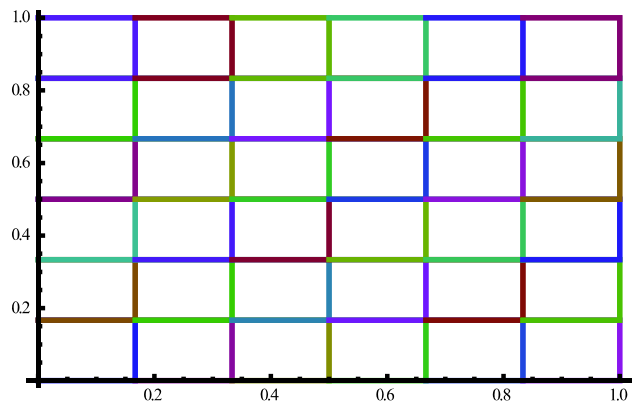

Fig. 4 Mesh for Convergence Tests

As shown in Fig. 5, the new GFEM offers improved accuracy and elevated convergence for an enlarged patch size; the convergence rates in both L_2 and energy norms increase one order in general when the patch size increases $1h$ (refer to the rate number in the convergence plots). The convergence property resembles the existing GFEM. *The most unique feature of the new GFEM is capable of offering different order of accuracy and convergence for the* ***same*** *mesh and the* ***same*** *number of dofs.* Normally this is a feature offered by a finite difference method, not readily shared by a FEM. In contrast, in the existing GFEM accuracy and convergence can only be improved through increasing nodes or extra dofs or the both.

Traditionally, we are not prone to consider a method efficient if it leads to a large nonzero bandwidth in its global (stiffness) matrix. This is particularly true in the PC/single core times. However, on the emerging architecture, in particular on manycore processors, the tunable sparsity would tell a different story. Tests follow.

8.4 Floating Point Performance on Emerging Architecture

The floating point performance test is focused on solving the system of linear equations associated with the new GFEM. Element assembly, as it is related to the definition of the local approximation, is out of consideration.

The global stiffness matrices of the new and the existing GFEMs are drawn for comparison for an 10-element mesh in 1D in Fig. 6. The sparsity of matrix is defined by the approximate ratio of nonzeros; the larger the ratio, the denser the matrix.

The differences in the global stiffness matrices between the two methods are that:

- as the order of local approximation increases, the global matrix does not vary in size in the new GFEM whereas it expands in the existing GFEM.
- as the order of local approximation increases, the matrix sparsity is reduced in the new GFEM whereas it nearly changes in the existing GFEM.
- the sparsity of the global matrix in the new GFEM is variable or *tunable* by varying patch size, a feature common to many meshfree methods.

The 2D problem of Eq. (8) is solved by the new GFEM. The calculation is carried out on both the CPU (using a single core) and the GPU to test the method's hardware-efficiency. The performance is measured in Gflop/s. Comparison is confined to the expense of solving the system of linear equations; the element stiffness assembly is out of consideration. The conjugate gradient method is used to solve the system of linear equations. The open source library CUSP [40] is used for the conjugating gradient method on the GPU. The sparse matrix is stored in the compressed sparse row (CSR) format. Test results are shown in Fig. 7.

First, we reduce the new GFEM to the standard FEM by letting $r = 0h$ and compare the sustained performance (Gflop/s) on the manycores of the GPU and the single core of the CPU. The performance data points for the FEM on the CPU and the GPU are circled in the figure. It is observed that the performance of the FEM has no obvious difference on

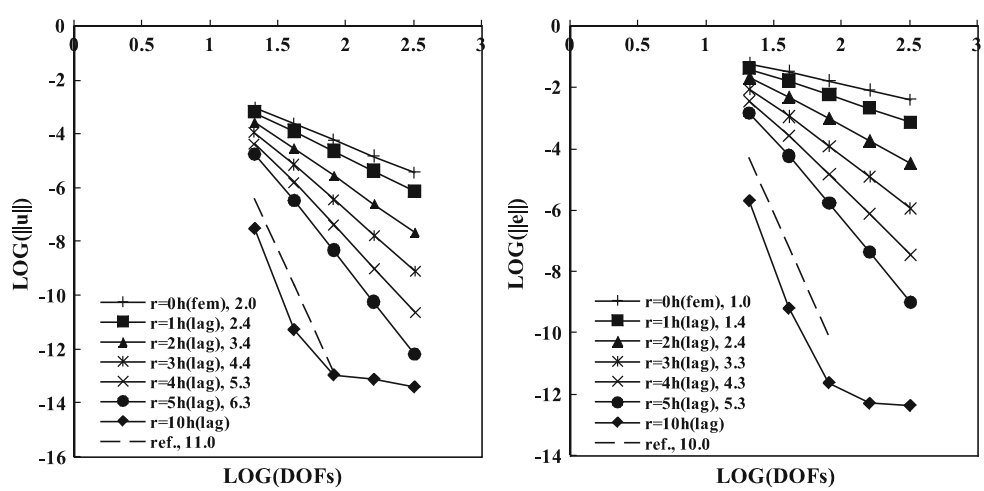

Fig. 5 Convergence test results. The tests show that the new GFEM is able to offer an elevated accuracy and convergence on the same mesh and the same number of dofs

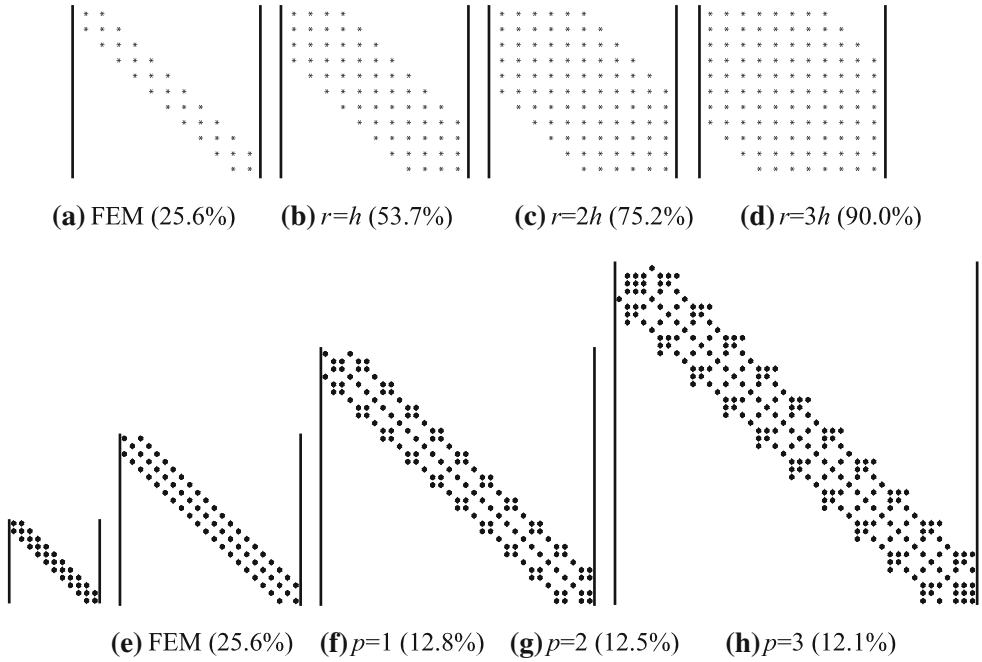

Fig. 6 Stiffness matrices on an 10-element mesh in 1D. **a**–**d** for the new GFEM, **e**–**h** for the existing GFEM. The number in *bracket* is the nonzeros percent, reflecting the sparsity of matrix. The existing GFEM shows nearly constant sparsity regardless of p while the new GFEM reduces the sparsity of matrix by increasing r

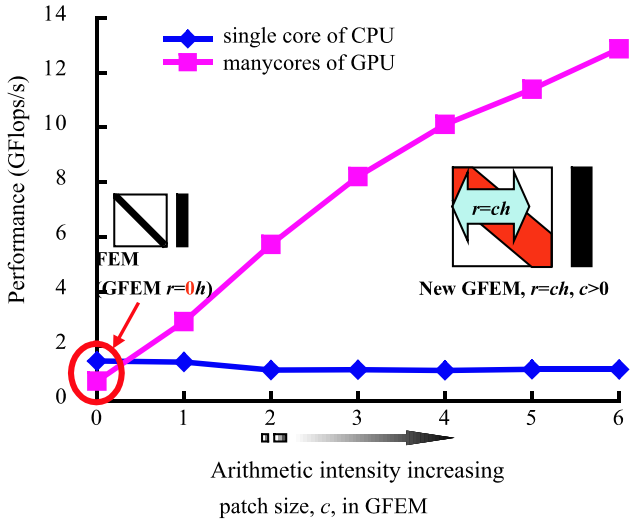

Fig. 7 Floating point performance of the new GFEM on manycores [19]

the single core of the CPU and the manycores of the GPU, although the GPU processor offers much higher floating point performance (peak performance: 78Gflop/s) than the single core CPU (peak performance: 10Gflop/s). This clearly shows that *the high floating point performance of the GPU is under-utilized if simply continuing to use the linear FEM on the manycore architecture*.

Second, we increase the patch size and measure the performance achieved on the CPU and the GPU. On the GPU, when increasing the patch size r, i.e., reducing the sparsity of the global stiffness matrix or increasing the order of the method, the performance increases significantly. This means the method well utilizes the high floating point performance of the manycores. This implies that *the new GFEM, although traditionally thought to be expensive from the classical viewpoint (in the PC/single core times), might be reservedly efficient on the manycore architecture*.

On the other hand, it is observed that on the CPU increasing the patch size does *not* result in better and even worsens the performance. This exactly explains the reason why the method leading to a dense matrix, or the high order method, has been traditionally deemed to be *not* computing efficient in the PC/single core times.

These tests, to a certain extent, reflect the change of what is "good" in the future: in the PC/single core times, memory is taken for granted and flop is expensive, whereas in the manycore times memory becomes the new bound while flop is free and even redundant. By tuning arithmetic intensity, the new GFEM can release the compute power of the manycores while offering the high order of accuracy and convergence.

What observed in the above should also apply to meshfree methods (refer to [41,42] for the detail of meshfree methods) because we are only focused on solving the system of linear equations. For both the new GFEM and the meshfree methods, their global stiffness matrices share one common feature that the stiffness matrix becomes dense for a high accuracy of approximation while the size of the system of linear equations not changing (like that shown in Fig. 6a–6d). Since the above tests are only focused on solving the system of linear equations, it is quite reasonable to believe that the meshfree method also shares the same hardware-efficiency on the manycore GPU in terms of solving the system of linear equations.

Noted is that element assembly for the meshfree/GFEM approximation is also expensive. However, as the element assembly is easy to parallelize and it is also method-dependent, the part of work leaves out of consideration in the paper. Two recent excellent references on this aspect can be found in [43,44].

8.5 Observations

Traditionally we are not prone to consider a method efficient when it leads to a moderately dense global (stiffness) matrix. This is particularly true in the PC/single core times. In the context of the new GFEM, the classical way of thinking is re-examined against the new computer architecture. The result reported in the paper is still preliminary. But it does reveal that the *co-designed* algorithm can well utilize the high floating point performance of the manycore GPU by the high order of accuracy and convergence; the "expensive" method is found to be reversely hardware-efficient.

9 Co-Design Example II: Mixed Precision Computing

9.1 The idea of mixed precision

Mixed precision in computation can increase the throughput of data movement (both memory access and network communication) while doubling flop/s. A mixed precision algorithm is to fully take advantage of low precision computation while not sacrificing the numerical accuracy of solution [45]. This has been shown advantageous on GPU [45–50], the Cell processor [51], and FPGA [47].

The idea of mixed precision can be traced back to Wilkinson in 1948 [52] using the iterative refinement (refer to [45,52–55]) to solve a system of linear equations.

The iterative refinement can be stated as

$$\begin{aligned}
&\mathbf{x}^{(0)} = \mathbf{0}\\
&\mathbf{d}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)} \quad \text{compute residual in } \textbf{\textit{high}} \text{ precision}\\
&\mathbf{A}\mathbf{c}^{(k)} = \mathbf{d}^{(k)} \quad \text{solve equation system in } \textbf{\textit{low}} \text{ precision}\\
&\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{c}^{(k)} \quad \text{accumulate solution in } \textbf{\textit{high}} \text{ precision}
\end{aligned}$$

Wilkinson and his collaborators [52] verified that if matrix $\mathbf{A}$ is not ill-conditioned, iterative refining $\mathbf{x}$ using mixed precision can reach the same precision done by fully high precision computation. For the above iterative refinement, 90 % float point operations can be done with low precision while not affecting the precision of solution. The effectiveness of the mixed precision calculation is dependent on whether there is precision loss. Buttari et al. [56] and Langou et al. [57] applied the mixed precision algorithm to dense matrices and verified the effectiveness of the mixed precision algorithm on Cell and other popular CPU architectures. Goddeke et al. [46–50] employed the iterative refinement in a multigrid method to solve a large system of linear equations. Those work showed that the solution obtained by using the mixed precision algorithm is the same as that obtained using fully high precision.

9.2 Mixed Precision Explicit FE Simulation

We extend the idea of mixed precision computation. The basic idea to be explored in this paper is *to work with deltas (the increment) in low precision and the whole value in high precision*. The idea is demonstrated by the following example.

Given a value of 1.000000000000001xxxxxxxx with a delta or small variation of "0.000000000000001xxxxxxxx", where "xxxxxxxx" stand for the decimal digits coming after. If working with the whole value, then we would have only *one* effective decimal digit to represent the small variation even in double precision. Instead, if we work with the delta, that is, 0.000000000000001xxxxxxxx or 1.xxxxxxxxE-15, instead of the whole value, then we have more effective decimal digits even in single precision. As long as the delta is small, working with the delta in low precision is reversely effective. If a numerical algorithm is re-designed in an incremental form, it may obtain high precision results using low precision computation.

To the best of the author's knowledge, the mixed precision idea is mostly applied to solving the system of linear equations (for examples [45–67]) and remains yet investigated in an explicit method. We have extended the idea to an explicit finite element method [5,68,69]. The mixed precision explicit FEM is described as follow:

$$\begin{aligned}
&\mathbf{m}\underset{(s)}{\Delta\mathbf{a}}^{k+1} = \beta\underset{(s)}{\Delta\mathbf{f}}^{k+1}\\
&\underset{(d)}{\mathbf{a}}^{k+1} = \underset{(d)}{\mathbf{a}}^{k} + \underset{(d)}{\Delta\mathbf{a}}^{k+1}/\beta\\
&\underset{(d)}{\mathbf{v}}^{k+1} = \underset{(d)}{\mathbf{v}}^{k} + \underset{(d)}{\mathbf{a}}^{k+1}\Delta t\\
&\underset{(d)}{\mathbf{u}}^{k+1} = \underset{(d)}{\mathbf{u}}^{k} + \underset{(d)}{\mathbf{v}}^{k+1}\Delta t
\end{aligned}$$

where (s) and (d) respectively denote the values computed using single and double precision. The basic idea of the above mixed precision algorithm is to solve an incremental form of Newton's second law using low precision and to update the acceleration using high precision to avoid precision loss. A scaling parameter β is introduced to avoid the underflow of deltas $\Delta\mathbf{a}$ and $\Delta\mathbf{f}$. β is chosen to be $||\mathbf{f}||_1/n$, where n is the length of the vector $\mathbf{f}$ [68,69].

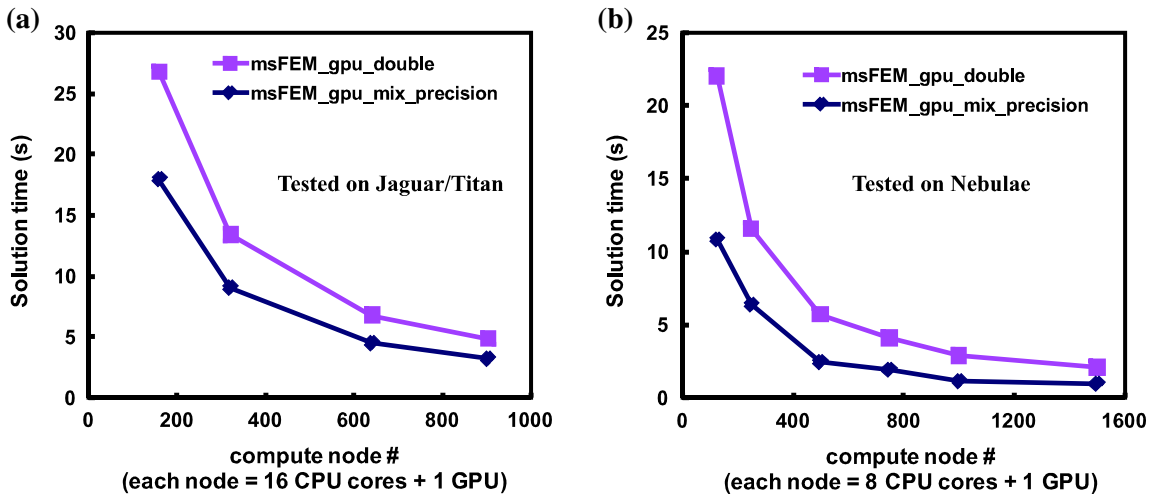

Fig. 8 Mixed precision explicit FEM. **a** Tested on Jaguar/Titan; **b** tested on Nabulae

9.3 A Mixed Precision FE Code

The mixed precision algorithm is implemented in a petascale simulation code msFEM. msFEM is an explicit code for multiscale analyses of material deformation, which is a 3D, parallel, nonlinear, large-deformation, multiscale code written in C++. The code is based on gradient plasticity theories [70–72] and the framework of generalized finite element methods [21–38,73–77]. The continuum model employed can be switched between the classical continuum model and the multi-resolution continuum model [78–86]. The generalized finite element framework allows an arbitrary number of dofs per node. The code has been used on BlueGene/L, Dawning 5000A, Nebulae and Jaguar/Titan.

msFEM has three different parallel implementations: (a) flat MPI version, msFEM_CPU; (b) CPU+GPU double precision version, msFEM_GPU_double; (c) CPU+GPU mixed precision version, msFEM_GPU_mixed. The three implementations are detailed as follows:

(a) msFEM_CPU: each CPU core runs a MPI process based on a traditional flat MPI programming model;
(b) msFEM_GPU_double, developed for GPU clusters. Each compute node runs a MPI process, the cores of CPU and GPU are collectively managed by multiple threads. It is based on a MPI+OpenMP+CUDA hybrid parallel programming model. All float point operations are done in double precision.
(c) msFEM_GPU_mixed, a mixed precision version of msFEM_GPU_double. The mixed precision is an acceleration strategy on vector units such as SSE and GPU using low precision for the majority of float point operations while maintaining the accuracy of the solution of application. The purpose of the mixed precision is to accelerate the time-to-solution by fully taking advantage of the compute power of low precision over high precision on the vector units. From the prospective of data movement, the use of low precision doubles the throughput of memory and network bandwidth.

9.4 Test Results

The test problem is a cantilever benchmark problem. The mesh model contains approximately 100 M linear tetrahedral elements. The meshes used for the tests on the two supercomputers are not exactly the same.

The GPU implementations have been tested on Jaguar/Titan (when Jaguar was being upgraded to Titan) and Nebulae. The test on Jaguar/Titan is done with Titan's GPU partition. Each compute node is equipped with one GPU processor. Each compute node runs one MPI process (therefore the number of MPI processes on each node is 1/16 of that of the flat MPI runs). One CPU core is assigned solely for data transfer and management and the rest 15 CPU cores are responsible for computation. The tests used up to 900 GPU nodes, which is almost the entirety of the GPU partition of Titan (the whole partition contains 960 GPU nodes). The test on Nebulae used 8 CPU cores co-processing with one GPU. The maximum number of GPU processors is 1,500. Test results are plotted in Fig. 8.

From the figure it is observed that the mixed precision version gains extra 1.5 times speedup over the double precision implementation.

In order to verify the accuracy of the mixed precision algorithm, the free vibration of the cantilever beam is analyzed respectively using double, mixed and single precision and computed up to 100, 000 steps, an amount of time steps that is large enough to justify whether the precision deterioration

Table 5 Comparisons of tip deflection obtained for double, mixed, and single precision

Step	Double precision	Mixed precision	Single precision
10,000	1.99746367211768e−01	1.99746367211764e−01	1.99746363787416e−01
20,000	−8.99992709756114e−02	−8.99992709756271e−02	−8.99992652988665e−02
30,000	6.57432245272338e−03	6.57432245270158e−03	6.57430622503326e−03
40,000	−1.95980520828321e−01	−1.95980520828340e−01	−1.95980508820425e−01
50,000	−2.08137323253383e−01	−2.08137323253380e−01	−2.08137322597563e−01
60,000	−2.60636338105768e−01	−2.60636338105777e−01	−2.60636275717600e−01
70,000	−3.05736878559848e−01	−3.05736878559853e−01	−3.05736838834926e−01
80,000	−2.41355171559992e−01	−2.41355171559980e−01	−2.41355162896363e−01
90,000	−2.28591319717476e−01	−2.28591319717470e−01	−2.28591253106985e−01
91,000	4.30101002316246e−02	4.30101002315829e−02	4.30100651415765e−02
92,000	1.34218167781684e−01	1.34218167781706e−01	1.34218179670874e−01
93,000	−3.25048239183419e−01	−3.25048239183450e−01	−3.25048209563430e-01
94,000	3.47672632081453e−01	3.47672632081454e−01	3.47672569972624e−01
95,000	−2.23039340016840e−01	−2.23039340016835e−01	−2.23039276399669e−01
96,000	6.02867763083984e−02	6.02867763083530e−02	6.02867478624269e−02
97,000	1.86595278347976e−01	1.86595278347996e−01	1.86595273470115e−01
98,000	−2.78869408234553e−01	−2.78869408234586e−01	−2.78869411873295e−01
99,000	2.62820038404892e−01	2.62820038404879e−01	2.62820006926713e−01
100,000	−1.23010090352524e−01	−1.23010090352522e−01	−1.23010059826073e−01

accumulates as the simulation goes. The tip deflection at the free end of the cantilever is obtained at different time steps for the double, the mixed, and the single precision, respectively, and is listed in Table 5. The output frequency is set to every 10,000 time steps for the first 90,000 steps and every 1,000 time step for the last 10,000 steps. At each time step, the result of the double precision is taken to be the best and the rest two are compared with it to count effective decimal digits. It is observed that the mixed precision algorithm maintains 10–14 effective decimal digits, where the single precision algorithm only maintains 5–8 effective decimal digits [68,69]. It is also shown that no error accumulation is observed even at the 100, 000th time step. This means that the precision loss does not accumulate and propagate as the simulation goes. The mixed precision leads to the almost same result of the double precision while accelerating simulation.

9.5 Observations

On the same hardware, the mixed precision algorithm offers extra 1.5 times speedup at the nearly same numerical accuracy. This speedup gains for two reasons: (a) doubled performance of single precision operations on GPU, (b) doubled throughput of data movement (improved usage of memory and network bandwidth). The speedup is significant by considering that *the mixed precision computing is faster than the fastest*—if taking "GPU computing in double precision" as the currently fastest way of computation.

10 Co-Design Example III: Concurrent/Flat Versus Hierarchical/Hybrid—Scalability is Compatibility

Petascale computing means to explore the concurrency of $O(10^6)$ while exascale computing is to challenge the concurrency of $O(10^9)$. Scalability and fault tolerance are two determining factors of a successful simulation at extreme-scale. We start our discussions from scalability tests of msFEM [124].

10.1 Petaflop/s Systems Used for Tests

Large-scale simulation and tests are made possible by the computing resource allocation awards on Jaguar and Nebulae, which are respectively No. 1 and No. 2 fast supercomputers on the Top500 list of June 2010 [6]. Jaguar is installed in the US DOE's Oak Ridge Laboratory, and Nebulae is located at the newly build National Supercomputing Centre in Shenzhen, China. Jaguar is built from Cray XT5-HE with Opteron 6-core processors. At the time of the tests, Jaguar was being upgraded to Titan with Opteron 16-core processors and NVidia Tesla C2090 GPUs. Nebulae is built from a Dawning TC3600 Blade system with Intel X5650 processors and NVidia Tesla C2050 GPUs.

10.2 Flat MPI Versus MPI+X

First, we test the performance of the flat MPI code, msFEM_CPU. Three mesh models are tested for the flat

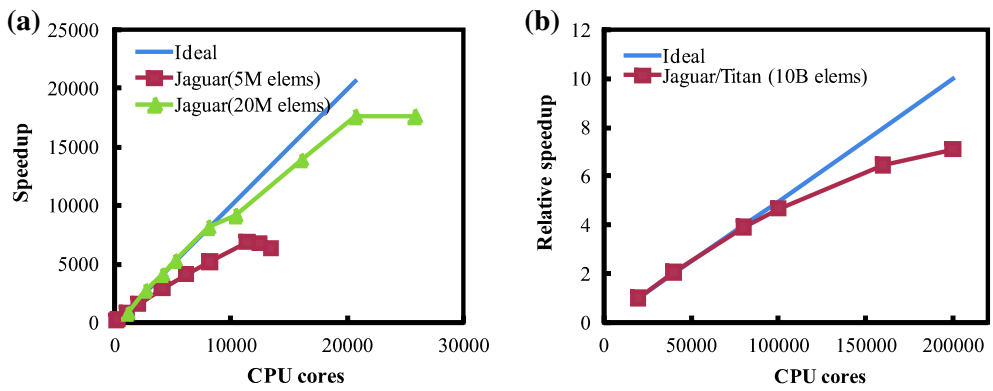

Fig. 9 Strong scalability of msFEM_CPU. **a** 2,500-20 K CPU cores; **b** 20 K–200 K CPU cores

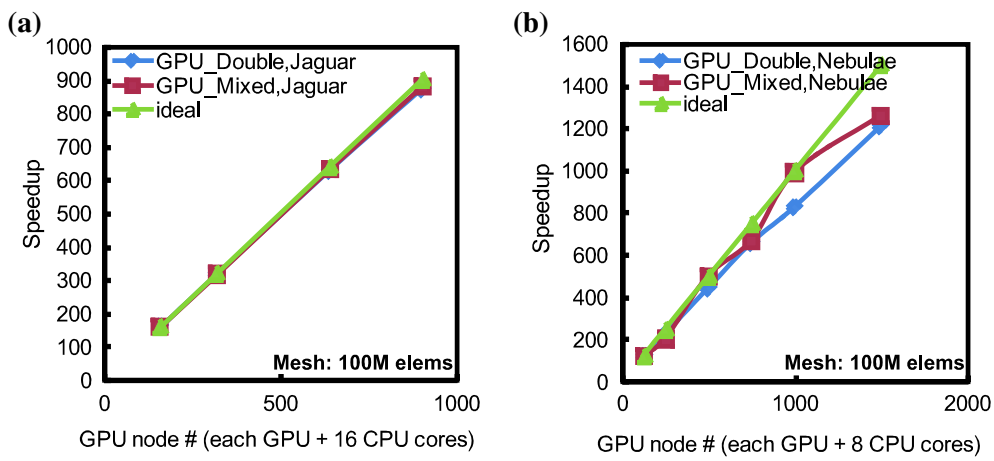

Fig. 10 Strong scalability tests of msFEM_GPU codes. **a** Jaguar; **b** Nebulae

MPI version, msFEM_CPU, which contain respectively 5 M, 20 M, and 2.7B tetrahedral elements. The first two mesh models have been used to test msFEM_CPU from 2500 to 20 K CPU cores and the largest mesh model was used to test msFEM_CPU from 20 K to 200 K CPU cores. The test results are plotted in Fig. 9.

It is seen that speedup saturates when the number of elements allocated to a CPU core is small and communication dominates. As the increase of the model size from 5M to 20 M elements, the speedup curve converges toward the ideal case, signaling a weak scalability of the msFEM code. With the largest mesh model, msFEM_CPU shows a reasonably good strong scalability up to 200 K CPU cores on Jaguar/Titan.

Next, we test the performance of the GPU versions. The GPU versions include the full precision and the mixed precision versions, i.e., msFEM_GPU_double and msFEM_GPU_mixed. The tests are performed on Jaguar/Titan and Nebulae. The mesh models used are approximately 100 M tetrahedral elements (but due to the availability of the computing facilities, the tests were done in different time and the meshes used for the two machines are not exactly the same). Test results are plotted in Fig. 10.

The tests in Figs. 9 and 10 are only different in the way of parallelization: the former follows a flat MPI parallel programming model while the latter is a MPI+X (X are pThreads + CUDA) hybrid parallel programming model. The implementations of the CPU-only version and the CPU+GPU co-processing version for the same numerical method and the same physical model enable a direct comparison between the two programming models, the flat and the hybrid models. It is clearly shown that the MPI+X hybrid model is much better scalable.

The good scalability is due to the increased grain size of parallelization enabled by the GPU accelerated compute node. But the more important reason, in the author's opinion, is the programming model matches the structure of hardware.

When moving from today's petaflop/s computers to tomorrow's exascale systems, we will expect a dramatic increase in concurrency. A billion ways of concurrency in conjunction with runtime imbalance of millions of compute

nodes [1], the flat MPI model will not scale down to use within future CPUs with hundreds or thousands of light-weight cores, in particular for those bulk synchronization intensive applications. For the billion ways of concurrency, the bulk synchronization is similar to stop the whole population of China for one single thing *and* to repeat the process frequently. Obviously this is hardly practical. Hence, the globally flat model of parallelism (e.g., the flat MPI) has to be structured into a hierarchy to coarsen the global synchronization. Computer architecture is converging at a heterogeneous structure of "cores + specialized cores + accelerators/coprocessors". The MPI+X hybrid parallel model follows the trend and hence offers the better scalability.

10.3 Observations

For the same reason of scalability, a numerical method may also be designed accordingly to fit into the structure of hardware. For example, a multiscale method should be implicit at coarse scale, which needs bulk synchronization only among a small set of compute nodes, while being explicit at fine scale which requires only intra-node local communication. And if judged by hardware compatibility, a hierarchical coupling method may be preferable over a concurrent coupling method. We see a natural match of the hybrid model of parallel programming with the hierarchical numerical method and a match of the flat model of parallel programming model with the concurrent numerical method. *The issue of scalability turns into the issue of compatibility among the model of parallel programming, the algorithm of numerical analysis, and the structure of hardware. This is indeed another example of co-design.*

11 Co-Design Example IV: Fault Tolerance, A Newly Emerging Problem that is Crucial to Simulation at Scale

As we move to petascale and exascale, the number of system components increases faster than component reliability [87–90,122]; faults will be regular rather than rare events [91,122]. This poses another challenge in extreme-scale computing: *a simulation hardly runs to the normal termination if not considering the unstable nature of the extreme-scale system.*

Currently, there is a debate whether checkpointing still works are not at extreme-scale [87,91] because to dump checkpoint data to disks may even take longer than mean time to failure (MTTF).

In the author's opinion, checkpointing works but requires domain experts to get involved. Time-dependent problems are among the most time-consuming applications. Fault tolerance is, relatively, more indispensable in the simulation of dynamic and unsteady problems than in static and steady problems. Therefore, dynamic and unsteady problems represent a typical scenario where fault tolerance is required.

A common feature of the simulation of dynamic or unsteady problems is to output result data every a certain number of time steps. Hence, the bottom line of hardware's stability for this kind of tough simulation is that the hardware is at least stable for a single step of output without abnormal termination—otherwise the hardware should not be considered ready for real applications. This roughly defines the lower bound of system reliability: the system's MTTF must be longer than the time to output the data of single time step.

In the simulation of dynamic and unsteady problems, a previous step of output already contains a majority of the data needed for checkpointing. This implies that the historical output data can be used for checkpointing. Because the output of the time-step data is the least requirement on system reliability, checkpointing becomes feasible.

The operating-system checkpointing treats an application code as a black box. But the way of checkpointing may lead to the overwhelming checkpoint data and a challenge in I/O. The is one of reasons that checkpointing may not work. It is domain experts who can assure a data-size optimal and light-weighted checkpoint. As such, domain experts have to get involved in developing the checkpointing capability. The interesting thing here is that the fault tolerance issue, which, no matter literally or traditionally, is a problem in the computer community, now needs a close collaboration between domain experts and computer scientists. This presents another excellent example of co-design.

11.1 petaPar: A Scalable and Fault Tolerant Particle Simulation Code

petaPar is a scalable and fault tolerant meshfree/particle simulation code, which is developed for petaflop/s supercomputers [92]. petaPar supports Smoothed Particle Hydrodynamics (SPH) [93–95] (the existing parallelization work of SPH can be found in [96–106]) and Material Point Method (MPM) [107–117,111,118], the two most mature and most powerful meshfree/particle methods. The developed code features:

Table 6 Test model

Parameter	Value	Process-wise value at 262,144 cores
Particles/material points	4,294,967,296	16,384
Patches	**64 × 64 × 64**	**1**
Cells	1024 × 1024 × 512	16 × 16 × 8
Points per cell	2 × 2 × 2	2 × 2 × 2
Cells per patch	16 × 16 × 8	16 × 16 × 8

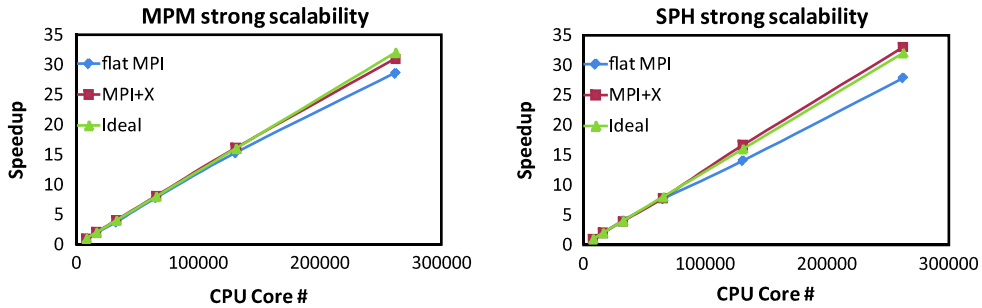

Fig. 11 Strong scalability of petaPar. MPM and SPH module linearly scale up to 260K CPU cores, the full system scale of Titan. The MPI+X hybrid version of each modulus delivers $90^{+}\%$ parallel efficiency and the SPH achieves super linear speedup

- adaptive to different hardware architectures by both the flat MPI and MPI+X parallel models;
- Highly scalable—largely overlapped communication by computation; dynamic load balanced.
- Highly fault tolerant—automatic self-starting from the historical output data using any number of MPI processes upon a local hardware failure such as inactive link, deadlock timeouts and even hard failure of nodes etc.

petaPar has been tested using a model of 4294967296 (4 billion of) particles up to 262,144 CPU cores on Titan. In petaPar, the computing task is organized in terms of patch. A patch is a regularly shaped cube in 3D composing of cells in which particles are bucketed. The test model contains$1024 \times 1024 \times 512$ cells. The cells are decomposed into $64 \times 64 \times 64$ patches. Each patch contains $16 \times 16 \times 8$ cells and each cell contains $2 \times 2 \times 2$ particles (see Table 6). At the scale of 262.144, the maximum scale tested, each CPU core only computes one patch or 16,384 particles. The strong scalability test results are shown in Fig. 11. It is observed that the code is linearly scalable. This good scalability is mainly due to the overlap of computation and communication.

11.2 Fault Tolerance Tests

petaPar's fault tolerance is made possible by automatic self-restart with a variable number of processes. Upon hardware failure such as inactive link, deadlock timeouts and machine check exception etc, the simulation can keep running with compute nodes available—different from the traditional design of restart, petaPar is designed to be able to restart with any number of processes so that it becomes fault tolerant.

The self-restart system includes a global control module *gctrl*, a gctrl command tool *gctrl_cmd*, a node monitoring module *dcrd* and the API library *libdcr* interfacing to the application code. *gctrl* has only one instance. *gctrl* registers new compute nodes, monitors the running of *dcrd*, and invokes the restart etc. *gctrl_cmd* configures and resets the restart command line and executes the restart command etc. *gctrl* starts *dcrd* on compute nodes. *dcrd* accepts the registration of local application process, monitors the status of the application process, and responds to the heart-breaking check of *gctrl*. *libdcr* interfaces to the application code to initialize and finalize the *dcrd*.

The application calls the initialization API function of *libdcr* to register itself in *dcrd*. When the application exits normally and calls the finalization API function of *libdcr*, *dcrd* also stops normally. When the application aborts abnormally and the finalization API is not called, *dcrd* can detect the abnormal abortion and notifies the *gctrl* to restart the application. The self-restart system is self-complete and independent.

We demonstrate the automatic self-restart process as follows:

1. Start *gctrl* with a list of hostnames (4 nodes, 48 cores):

```
gctrl -f ~/restart/host
```

2. Register the restart command line in *gctrl_cmd*.

```
gctrl_cmd -f ./restart
```

3. Run the app "petaPar" using two nodes (the second figure shown screen outputs):

```
nohup mpiexec -f host -np 16 ./petaPar mpm ./input.ict&
```

```
app connect: petaPar
app host: nth=0,host=blade01
app host: nth=1,host=blade02
```

4. Manually kill petaPar processes.

```
killall petaPar
```

gctrl detects the abortion of the simulation process and automatically execute the restart command using the compute resource registered:

```
broadcast to kill app process
restart app now
restart: cd /home/lils/petapar && nohup ~/mpich/bin/mpiexec -f /tmp/.dcrd3.hosts
-np 48 /home/lils/petapar/petaPar mpm input.ict input 2>&1>>/dev/null&
app connect: petaPar
app host: nth=1,host=blade02
app host: nth=3,host=blade03
app host: nth=0,host=blade01
app host: nth=2,host=blade04
```

5 Stop the *dcrd* on one of nodes to simulate a nodal failure.

```
killall dcrd
```

6 Then *gctrl* detects the connection loss to the node and invokes a restart using the rest available nodes. In the case, only 3 nodes or 36 cores are available for the restart:

```
blade01 loss connection
broadcast to kill app process
restart app now
restart: cd /home/lils/petapar && nohup ~/mpich/bin/mpiexec -f /tmp/.dcrd3.hosts
-np 36 /home/lils/petapar/petaPar mpm input.ict input 2>&1>>/dev/null&
app connect: petaPar
app host: nth=1,host=blade03
app host: nth=3,host=blade04
app host: nth=2,host=blade02
```

7. Restart *dcrd* on the node that just killed, to simulate the add-in of a new compute node. Then *gctrl* detects the change of computing resources and restart the application again using the newly available computer resources:

```
dcrd -g blade01
gctrl_cmd -r petaPar
restart command for petaPar
```

```
dcrd connect: host=blade01,cores=12
broadcast to kill app process
restart app now
restart: cd /home/lils/petapar && nohup ~/mpich/bin/mpiexec -f /tmp/.dcrd3.hosts
-np 48 /home/lils/petapar/petaPar mpm input.ict input 2>&1>>/dev/null&
app connect: petaPar
app host: nth=1,host=blade02
app host: nth=3,host=blade01
app host: nth=0,host=blade03
app host: nth=2,host=blade04
```

With the self-restart function, petaPar is able to run continuously in a dynamic environment. Although large-scale tests of the function is still undergoing and soft error is currently out of consideration, we believe that the automatic self-restart from variable number of MPI processes is good enough to deal with fault like inactive links, connection timouts and even hard failure of compute nodes.

11.3 Observations

Simply put, a product simulation cannot be successfully done if we do not take into consideration frequent failures on the fly. As such, the development of an extreme-scale simulation code would not be complete if without being equipped with a fault tolerance function. The fault tolerance issue, which is an issue completely beyond the mind of an application developer in the past, becomes a must to do nowadays. Domain experts need to get involved to assure the data-size of checkpoint to be optimal, which again makes a co-design necessary.

12 Conclusions

As the power of modern supercomputing systems continues to advance at an exciting pace forward to extreme scales, it is quite clear that the associated software development challenges are also increasingly formidable. Due to the fact that the memory intensive S&E applications cannot simply benefit from the increasing number of cores, the "performance gap" has a tendency to become wider on the emerging manycore architecture. In the community of computer science, a motion being pushed forward is to *co-design* between numerical algorithms with the emerging computer architecture [1,2,4,5,119,120].

Co-design, in the most basic sense, engages the necessary collaborations between hardware designers, computer scientists, applied mathematicians, and computational science experts in multiple application domains to carry out the essential interdisciplinary research that will enable in a timely way the scientific and technological benefits as HPC hardware moves forward to exascale.

From the prospective of the co-design, the author by this review paper suggested some new ways of thinking when selecting, optimizing, and developing a numerical method and a simulation tool to meet the challenge of simulation at scale. We conclude the review paper by citing what John Shalf, Sudip Dosanjh, and John Morrison have written in [3]: "*there is an unprecedented opportunity for application and algorithm developers to influence the direction of future (computer) architectures*".

Acknowledgments Zedong Wu helped to calculate the data of Fig. 7, Chaowei Wang helped with the implementation of the mixed precision algorithm of msFEM, Jianhua Liu and Jiangyong Ren helped with Figs. 8, 9, and 10. Leisheng Li helped to parallelize petaPar and to obtain the data of Fig. 11. Fuxi Zhang and Zhigang Huo coded *dcrd*. The research described in this paper was financially supported by the "100 Talent Program" of Chinese Academy of Sciences and the National Foundation of Sciences of China (Grand numbers: 11072241, 11111140020, 91130026, 60633040). This research used resources of the Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory, which is supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC05-00OR22725. This research used resources of the National Supercomputing Centre in Shenzhen, China.

References

1. DOE Exascale Initiative Roadmap (2009) Architecture and technology workshop. San Diego
2. DOE Office of Science Summary report of the Advanced Scientific Computing Advisory Committee (ASCAC) Subcommittee. The opportunities and challenges of exascale computing, Fall 2010
3. Shalf J, Dosanjh S, Morrison J (2011) Exascale computing technology challenges. VECPAR 2010. LNCS 6449:1–25
4. Tian R, Sun N (2013) Some considerations about exascale computing in China. Commun China Comput Fed 9(2):52–60 (in Chinese)
5. Tian R (2013) Co-design thinking towards exascale computing. Inf Technol Lett 10(3):50–63 (in Chinese)
6. http://www.top500.org/project/linpack/
7. Thibodeau P (2012) Exascale unlikely before 2020 due to budget woes. Computerworld. Nov 19, 2012
8. Harrod W (2012) DOE exascale computing Initiative (ECI) update. DOE, Office of Science (SC), Oct 4, 2012
9. Dongarra J (2013) Emerging heterogeneous technologies for high performance computing. 22nd International Heterogeneity in Computing Workshop. IPDP, Boston
10. DOE E3 Report, http://www.er.doe.gov/ascr/ProgramDocuments/ProgDocs.html
11. A platform, strategy for the advanced simulation and computing, Program (NA-ASC-113R-07-Vol. 1-Rev. 0)
12. http://www.nccs.gov/
13. Chen J, Bell J (2011) Combustion exascale co-design center. Sixth international exascale software project workshop, San Francisco, April 6–7
14. Dennis JM, Edwards J, Guba O, St-Cyr A, Taylor MA, Worley PH, (2012) CAM-SE: a scalable spectral element dynamical core for the community atmosphere model. Int J High Perform Comput Appl 26(1):74–89
15. http://lammps.sandia.gov/
16. http://ees.lanl.gov/pflotran/
17. Eisenbach M, Zhou CG, Nicholson DM, Brown G, Larkin J, and Schulthess TC (2010) Thermodynamics of magnetic systems from first principles: WL-LSMS. In the proceeding of the 52nd Cray User Group meeting, CUG 2010
18. http://nvworld.ru/files/articles/calculations-on-gpu-advantages-fermi/fermipeformance.pdf
19. Tian R (2013) Meshfree/GFEM in hardware-efficiency prospective. Interaction and multiscale mechanics. DOI:10.12989/imm.2013.6.2.000
20. Tian R (2013) Extra-dof-free and linearly independent enrichments in GFEM. Comput Method Appl Mech Eng 266:1–22
21. Babuška I, Melenk JM (1997) Partition of unity method. Int J Numer Method Eng 40:727–758
22. Melenk JM, Babuška I (1996) The partition of unity finite element method: basic theory and applications. Comput Method Appl Mech Eng 139:289–314
23. Babuška I, Caloz G, Osborn JE (1994) Special finite element methods for a class of second order elliptic problems with rough coefficients. SIAM J Numer Anal 31:945–981
24. Duarte CA, Oden JT (1996) An h-p adaptive method using clouds. Comput Methods Appl Mech Eng 139(1–4):237–262
25. Oden JT, Duarte CA, Zienkiewicz OC (1998) A new cloud-based hp finite element method. Comput Method Appl Mech Eng 153(1–2):117–126
26. Strouboulis T, Babuška I, Copps K (2000) The design and analysis of the generalized finite element method. Comput Method Appl Mech Eng 181(1–3):43–69

27. Strouboulis T, Copps K, Babuška I (2000) The generalized finite element method: an example of its implementation and illustration of its performance. Int J Numer Method Eng 47:1401–1417
28. Strouboulis T, Copps K, Babuška I (2001) The generalized finite element method. Comput Method Appl Mech Eng 190(32–33):4081–4193
29. Strouboulis T, Zhang L, Babuška I (2003) Generalized finite element method using mesh-based handbooks: application to problems in domains with many voids. Comput Method Appl Mech Eng 192:3109–3161
30. Strouboulis T, Zhang L, Babuška I (2004) p-version of the generalized FEM using mesh-based handbooks with applications to multiscale problems. Int J Numer Method Eng 60:1639–1672
31. Strouboulis T, Zhang L, Wang D, Babuška I (2006) A posteriori error estimation for generalized finite element methods. Comput Method Appl Mech Eng 195:852–879
32. Strouboulis T, Babuška I, Hidajat R (2006) The generalized finite element method for Helmholtz equation: theory, computation, and open problems. Comput Method Appl Mech Eng 195:4711–4731
33. Strouboulis T, Hidajat R, Babuška I (2008) The generalized finite element method for Helmholtz equation, part II: effect of choice of handbook functions, error due to absorbing boundary conditions and its assessment. Comput Method Appl Mech Eng 197:364–380
34. Duarte CA, Babuška I, Oden JT (2000) Generalized finite element methods for three-dimensional structural mechanics problems. Comput Struct 77:215–232
35. Duarte CA, Hamzeh ON, Liszka TJ, Tworzydlo WW (2001) A generalized finite element method for the simulation of three-dimensional dynamic crack propagation. Comput Method Appl Mech Eng 190:2227–2262
36. Simone A, Duarte CA, Van der Giessen E (2006) A generalized finite element method for polycrystals with discontinuous grain boundaries. Int J Numer Method Eng 67:1122–1145
37. Duarte CA, Kim DJ (2008) Analysis and applications of a generalized finite element method with global-local enrichment functions. Comput Method Appl Mech Eng 197(6–8):487–504
38. O'Hara P, Duarte CA, Eason T (2009) Generalized finite element analysis for three dimensional problems exhibiting sharp thermal gradients. Comput Method Appl Mech Eng 198:1857–1871
39. Lancaster P, Salkauskas K (1981) Surfaces generated by moving least squares methods. Math Comput 37:141–158
40. http://code.google.com/p/cusp-library/
41. Belytschko T, Lu YY, Gu L (1994) Element-free Gakerkin method. Int J Numer Method Eng 37:229–256
42. Li S, Liu WK (2001) Meshfree and particle methods and their applications. Appl Mech Rev 55:1–34
43. Cecka C, Lew A, Darve E (2011) Assembly of finite element methods on graphics processors. Int J Numer Method Eng 85(5):640–669
44. Karatarakis A, Metsis P, Papadrakakis M (2013) GPU-acceleration of stiffness matrix calculation and efficient initialization of EFG meshless methods. Comput Method Appl Mech Eng (in press), Accepted Manuscript, Available online 4 March 2013
45. Buttari A, Dongarra J, Kurzak J, Luszczek P, Tomov S (2008) Using mixed precision for sparse matrix computations to enhance the performance while achieving 64-bit accuracy. ACM Trans Math Softw (TOMS) 34(4):1–22
46. Göddeke D, Strzodka R, Turek S (2005) Accelerating double precision FEM simulations with GPUs. In Proceedings of ASIM 2005–18th symposium on simulation technique
47. Strzodka R, Göddeke D (2006) Pipelined mixed precision algorithms on FPGAs for fast and accurate PDE solvers from low precision components. In IEEE symposium on field-programmable custom computing machines (FCCM 2006), pp 259–268
48. Strzodka R, Göddeke D (2006) Mixed precision methods for convergent iterative schemes. In Proceedings of the 2006 workshop on edge computing using new commodity architectures, pp D-59-60, May 2006
49. Göddeke D, Strzodka R, Turek S (2007) Performance and accuracy of hardware-oriented native-, emulated- and mixed-precision solvers in FEM simulations. Int J Parallel, Emergent Distrib Syst (IJPEDS), Special issue: Appl. Parallel Comput 22(4):221–256
50. Göddeke D, Strzodka R (2008) Performance and accuracy of hardware-oriented native-, emulated- and mixed-precision solvers in FEM simulations (part 2: Double precision GPUs). Technical University Dortmund, Technical report
51. Jakub Kurzak, Jack Dongarra (2007) Implementation of mixed precision in solving systems of linear equations on the Cell processor. Concurr Comput Pract Experience 19(10):1371–1385
52. Wilkinson JH (1963) Rounding errors in algebraic processes. Prentice-Hall, Englewood Cliffs
53. Moler CB (1967) Iterative refinement in floating point. J ACM 14(2):316–321
54. Jankowski M, Woniakowski H (1977) Iterative refinement implies numerical stability. J BIT Numer Math 17(3):303–311
55. Higham NJ (2002) Accuracy and stability of numerical algorithms. Society for Industrial and Applied Mathematics, Philadelphia
56. Buttari A, Dongarra J, Langou J, Langou J, Luszczek P, Kurzak J (2007) Mixed precision iterative refinement techniques for the solution of dense linear systems. Int J High Perform Comput Appl 21:457–466
57. Langou J, Langou J, Luszczek P, Kurzak J, Buttari A, Dongarra J (2006) Exploiting the performance of 32 bit floating point arithmetic in obtaining 64 bit accuracy (revisiting iterative refinement for linear systems). Proceedings of the 2006 ACM/IEEE conference on supercomputing
58. Demmel JW (1997) Applied numerical linear algebra. SIAM Press, Philadelphia
59. Demmel J, Hida Y, Kahan W, Li XS, Mukherjee S, Riedy EJ (2005) Error bounds from extra precise iterative refinement. Technical Report No. UCB/CSD-04-1344, LAPACK Working Note 165, Feb 2005
60. Taiji M, Narumi T, Ohno Y, Futatsugi N, Suenaga A, Takada N, Konagaya A (2003) Protein explorer: a petaflops special-purpose computer system for molecular dynamics simulations. Proceedings of Supercomputing 2003 in CD-ROM
61. Anderson E, Bai Z, Bischof C, Blackford LS, Demmel JW, Dongarra JJ, Du Croz J, Greenbaum A, Hammarling S, McKenney A, Sorensen D LAPACK Users' Guide. SIAM, http://www.netlib.org/lapack/
62. Li XS, Demmel JW, Bailey DH, Henry G, Hida Y, Iskandar J, Kahan W, Kang SY, Kapur A, Martin MC, Thompson BJ, Tung T, Yoo DJ (2002) Design, implementation and testing of extended and mixed precision BLAS. ACM Trans Math Softw (TOMS) 28(2):152–205
63. Göddeke D, Strzodka R, Turek S (2007) Performance and accuracy of hardware-oriented native-, emulated- and mixed-precision solvers in FEM simulations. Int J Parallel Emergent Distrib Syst (Special Issue: Applied Parallel Computing) 22(4):221–256
64. Göddeke D, Wobker H, Strzodka R, Mohd-Yusof J, McCormick P, Turek S (2009) Co-processor acceleration of an unmodified parallel solid mechanics code with FEASTGPU. Int J Comput Sci Eng 4(4):254–269
65. Strzodka R, Göddeke D (2006) Pipelined mixed precision algorithms on FPGAs for fast and accurate PDE solvers from low precision components. In FCCM'06: Proceedings of the 14th annual IEEE symposium on field-programmable custom computing machines (FCCM'06) pp 259–270

66. Strzodka R, Göddeke D (2006) Mixed precision methods for convergent iterative schemes. In Proceedings of the 2006 workshop on edge computing using new commodity architectures, D-59-60
67. Kurzak J, Dongarra JJ (2007) Implementation of mixed precision in solving systems of linear equations on the CELL processor. Concurr Comput Pract Experience 19(10):1371–1385
68. Jianhua Liu, Chaowei Wang, Jiangyong Ren, Rong Tian (2012) A mixed precision explicit finite element algorithm on heterogeneous architecture and its CUDA implementation. Comput Sci 39(6):293–296 (in Chinese)
69. Liu J (2011) A mixed precision GPU acceleration algorithm and its application to FEM. MS thesis of Graduate School of Chinese Academy of Sciences (in Chinese)
70. Aifantis EC (1992) On the role of gradients in the localization of deformation and fracture. Int J Eng Sci 30(10):1279–1299
71. Hill R (1963) Elastic properties of reinforced solids: some theoretical principles. J Mech Phys Solids 11(5):357–372
72. Hill R (1972) On constitutive macro-variables for heterogeneous solids at finite strain. Proc R Soc Lond Ser A Math Phys Sci 326(1565):131–147
73. Tian R, Yagawa G (2005) Generalized node and high-performance elements. Int J Numer Method Eng 64:2039–2071
74. Tian R, Yagawa G, Terasaka H (2006) Linear dependence problems of partition of unity based generalized FEMs. Comput Method Appl Mech Eng 195:4768–4782
75. Tian R (2006) A PU-based 4-node quadratic tetrahedron and linear dependence elimination in three dimensions. Int J Comput Method 3:545–562
76. Tian R, Matsubara H, Yagawa G (2006) Advanced 4-node tetrahedrons. Int J Numer Methods Eng 68:1209–1231
77. Tian R, Yagawa G (2006) Allman's triangle, rotational dof and partition of unity. Int J Numer Method Eng 69:837–858
78. McVeigh C, Liu WK (2010) Multiresolution continuum modeling of micro-void assisted dynamic adiabatic shear band propagation. J Mech Phys Solid 58(2):187–205
79. McVeigh C, Vernerey F, Liu WK, Brinson C (2006) Multiresolution analysis for material design. Comput Method Appl Mech Eng 195:5053–5076
80. McVeigh C, Vernerey FJ, Liu WK, Moran B, Olson GB (2007) An Interactive microvoid shear localization mechanism in high strength steels. J Mech Phys Solids 55(2):224–225
81. McVeigh C (2007) Ph.D. Thesis, Northwestern University
82. McVeigh C, Liu WK (2008) Linking microstructure and properties through a predictive multiresolution continuum. Comput Method Appl Mech Eng 197:3268–3290
83. McVeigh C, Liu WK (2009) Multiresolution modeling of ductile reinforced brittle composites. J Mech Phys Solids 57:244–267
84. Tian R, Moran B, Liu WK, Olson GB (2008) Multiscale fracture simulator. Dynamic microstructure design consortium (ONR Contract: N00014–05-C-0241) base final Report
85. Tian R, Liu WK, Chan S, Olson GB, Tang S, Wang JS, Jou HJ, Gong JD, Moran B (2009) Linking microstructures to fracture toughness-predictive 3D process zone simulations. The D 3-D annual PI Review, Evanston, March 23–25
86. Rong Tian, Stephanie Chan, Shan Tang, Kopacz Adrian M, Jian-Sheng Wang, Herng-Jeng Jou, Larbi Siad, Lars-Erik Lindgren, Gregory Olson, Kam Liu Wing (2010) A multi-resolution continuum simulation of the ductile fracture process. J Mech Phys Solids 58(10):1681–1700
87. Dongarra J et al The international exascale software project roadmap. www.iesp.org
88. Schroeder B, Gibson GA (2006) A large-scale study of failures in high-performance computing systems. Proceedings of the international conference on dependable systems and networks pp 249–258
89. Liu Y (2007) Reliability-aware optimal checkpoint/restart model in high performance computing, PhD Thesis. Louisiana
90. Cappello F, Geist A, Gropp B et al (2009) Toward exascale resilience. Int J High Perform C 23:374–388
91. Geist A (2009) Co-design challenges going from petascale to exascale. Workshop on bio-molecular simulations on future computing architectures, Oak Ridge
92. Li L, Wang C, Ma Z, Tian R (2013) petaPar: a highly scalable and fault tolerant meshfree/particle simulation code based on free assembly mesh. HPC China 2013, Guilin, China, October 29–31, 2013
93. Gingold RA, Monaghan JJ (1977) Smoothed particle hydrodynamics: theory and application to non-spherical stars. Mon Not R Astron Soc 181:375–389
94. Libersky LD, Petschek AG (1990) Smooth particle hydrodynamics with strength of materials. Adv Free Lagrange Method Lect Notes Phys 395:248–257
95. Liu MB, Liu GR (2010) Smoothed particle hydrodynamics (SPH): an overview and recent developments. Arch Comput Method Eng 17:25–76
96. Warren MS, Salmon JK (1995) A portable parallel particle program. Comput Phys Commun 87(1):266–290
97. Goozee RJ, Jacobs PA (2003) Distributed and shared memory parallelism with a smoothed particle hydrodynamics code. Anziam J 44:202–228
98. Maruzewski P, TouzéD L, Oger G et al (2010) SPH high-performance computing simulations of rigid solids impacting the free-surface of water. J Hydraul Res 48(S1):126–134
99. Springel V (2005) The cosmological simulation code gadget-2. Mon Not R Astron Soc 364(4):1105–1134
100. Holmes DW, Williams JR, Tilke P (2011) A framework for parallel computational physics algorithms on multi-core: SPH in parallel. Adv Eng Softw 42(11):999–1008
101. Ihmsen M, Akinci N, Becker M et al (2011) A Parallel SPH Implementation on Multi-Core CPUs. Comput Graph Forum 30(1): 99–112
102. Harada T, Koshizuka S, Kawaguchi Y (2007), Smoothed particle hydrodynamics on GPUs. Proc Comput Graph Int pp 63–70
103. Hérault A, Bilotta G, Dalrymple RA (2010) SPH on GPU with CUDA. J Hydraul Res 48(S1):74–79
104. Valdez-Balderas D, Domínguez J M, Rogers BD, et al. (2012) Towards accelerating smoothed particle hydrodynamics simulations for free surface flows on multi-GPU clusters. J Parallel Distr Com
105. Domínguez JM, Crespo AJC, Valdez-Balderas D et al (2013) New multi-GPU implementation for smoothed particle hydrodynamics on heterogeneous clusters. Comput Phys Commun 184:1848–1860
106. Domínguez JM, Crespo AJC, Gómez-Gesteira M (2013) Optimization strategies for CPU and GPU implementations of a smoothed particle hydrodynamics method. Comput Phys Commun 184:617–627
107. Sulsky D, Chen Z, Schreyer HL (1994) A particle method for history-dependent materials. Comput Method Appl Mech Eng 118:179–196
108. Love E, Sulsky DL (2006) An unconditionally stable, energy-momentum consistent implementation of the material-point method. Comput Method Appl Mech Eng 195(33–36):3903–3925
109. Wallstedt PC, Guilkey JE (2008) An evaluation of explicit time integration schemes for use with the generalized interpolation material point method. J Comput Phys 227(22):9628–9642
110. Zhang Duan Z, Xia Ma, Giguere Paul T (2011) Material point method enhanced by modified gradient of shape function. J Comput Phys 230(16):6379–6398

111. Więckowski Z (2004) The material point method in large strain engineering problems. Comput Method Appl Mech Eng 193(39–41):4417–4438
112. Sulsky D, Kaul A (2004) Implicit dynamics in the material-point method. Comput Method Appl Mech Eng 193(12–14):1137–1170
113. Wang HK, Liu Y, Zhang X (2012) The carbon nanotube composite simulation by material point method. Comput Mater Sci 57:23–29
114. Zhang X, Sze KY, Ma S (2006) An explicit material point finite element method for hyper velocity impact. Int J Numer Method Eng 66:689–706
115. Lian YP, Zhang X, Liu Y (2012) An adaptive finite element material point method and its application in extreme deformation problems. Comput Method Appl Mech Eng 241–244:275–285
116. Lian YP, Zhang X, Liu Y (2011) Coupling of finite element method with material point method by local multi-mesh contact method. Comput Method Appl Mech Eng 200(47–48):3482–3494
117. Więckowski Z (2004) The material point method in large strain engineering problems. Comput Method Appl Mech Eng 193(39–41):4417–4438
118. Sulsky D, Kaul A (2011) Implicit dynamics in the material-point method. Comput Method Appl Mech Eng 193(12–14):1137–1170
119. http://science.energy.gov/ascr/research/scidac/co-design/
120. http://www.ncic.ac.cn/codesign
121. Joubert W (2012) Porting the denovo radiation transport code to Titan: lessons learned. OLCF Titan Workshop 2012
122. Franck Cappello (2009) Fault tolerance in petascale/ exascale systems: current knowledge, challenges and research opportunities. Int J High Perform Comput Appl 23:212–226
123. Keyes D (2012) Large-scale simulation in science and engineering: digesting the fruit, replanting the fields. Co-Design 2012, Beijing, China, October 23–25, 2012
124. Ren J, Wang CW, Wang Y, Tian R (2013) Scalability tests of a finite element code on hundreds of thousands cores and heterogeneous architecture. Comm Comp Info Sci 207:151–165

View publication stats